

Apptainer Container Guide

CSE Computation Cluster

Version 1.0

April 2025

Contents

1	Introduction	3
1.1	Two Main Approaches	3
1.2	Why Apptainer for Cluster?	3
2	Method 1: Pulling from a Container Registry (Simplest)	3
2.1	Basic Pull Command	3
2.2	Examples for Your Cluster	3
3	Method 2: Building from a Definition File (Custom)	4
3.1	Definition File Structure	4
4	Example Definition Files	4
4.1	Example 1: Simple Ubuntu with Python	4
4.2	Example 2: GPU-Enabled PyTorch Container (For Your Cluster)	5
4.3	Example 3: Conda/Python Environment	5
4.4	Example 4: R Environment for Data Science	6
4.5	Example 5: Complete ML Environment (PyTorch + TensorFlow + Jupyter)	6
5	Build Commands Reference	7
5.1	Build Options Explained	7
6	Transferring to Your Cluster	8
6.1	Organizing Containers on the Cluster	8
7	Running container on the Cluster	8
7.1	Basic Slurm Job Script with Container	8
7.2	Container Flags Explained	8
7.3	Example: Mounting Directories	9
7.4	Example: Different Container Types	9

8	Troubleshooting	10
8.1	Common Errors and Solutions	10
8.2	Cleaning Cache	10
9	Best Practices	10
10	Quick Reference Card	10
11	Quick Checklist Before Building	11

Introduction

Apptainer (formerly Singularity) is the standard container solution for cluster environments. This guide explains how to create and use custom container images.

1.1 Two Main Approaches

Approach	Best For	Customization	Command
Pull from Registry	Quick start, standard tools	Limited	<code>apptainer build <output.sif> docker://<image></code>
Build from Definition	Custom environments, specific software	Full	<code>apptainer build <output.sif> <definition.def></code>

1.2 Why Apptainer for Cluster?

- **No root required** - Containers run as your user
- **Native GPU support** - Use `--nv` flag for CUDA
- **Single file images** - Easy to transfer (.sif files)
- **Slurm integration** - Works seamlessly with job scripts

Recommendation: Build the apptainer image on your local PC (where you have `sudo/fakroot`) and transfer the .sif file to your submission node.

Method 1: Pulling from a Container Registry (Simplest)

This is the quickest way to get a container. Apptainer can pull images from Docker Hub and other registries.

2.1 Basic Pull Command

```
1 apptainer build mycontainer.sif docker://ubuntu:24.04
```

2.2 Examples for Your Cluster

Purpose	Command
PyTorch GPU	<code>apptainer build pytorch.sif docker://pytorch/pytorch:2.11.0-cuda13.0-cudnn9-devel</code>
TensorFlow	<code>apptainer build tensorflow.sif docker://tensorflow/tensorflow:latest-gpu</code>

CUDA Base	apptainer build cuda.sif docker://nvidia/cuda:12.1.1-devel-ubuntu22.04
Ubuntu Base	apptainer build ubuntu.sif docker://ubuntu:24.04

Method 2: Building from a Definition File (Custom)

For custom software installations, create a **definition file** (recipe) and build from it.

3.1 Definition File Structure

A .def file has a header followed by optional sections:

```

1 # Header - specifies base image
2 Bootstrap: docker
3 From: ubuntu:24.04
4
5 # %post - runs at build time (install software)
6 %post
7     apt-get update
8     apt-get install -y python3 python3-pip
9
10 # %environment - sets runtime environment variables
11 %environment
12     export PATH=/usr/local/bin:$PATH
13
14 # %runscript - default command when running container
15 %runscript
16     echo "Container starting..."
17     exec "$@"
18
19 # %labels - metadata
20 %labels
21     Author Your Name
22     Version 1.0
23
24 # %help - documentation shown with 'apptainer run-help'
25 %help
26     This container runs Python 3 on Ubuntu 24.04

```

Example Definition Files

4.1 Example 1: Simple Ubuntu with Python

Create python.def:

```

1 Bootstrap: docker
2 From: ubuntu:24.04
3
4 %post
5     apt-get update
6     apt-get install -y python3 python3-pip git
7     pip3 install numpy pandas matplotlib
8     apt-get clean
9

```

```

10 %environment
11     export LC_ALL=C
12
13 %runscript
14     python3 "$@"

```

Build and test:

```

1 apptainer build python.sif python.def
2 apptainer exec python.sif python3 -c "import numpy; print(numpy.
    __version__)"

```

4.2 Example 2: GPU-Enabled PyTorch Container (For Your Cluster)

Create pytorch-gpu.def:

```

1 Bootstrap: docker
2 From: nvidia/cuda:12.1.1-devel-ubuntu22.04
3
4 %post
5     # System updates
6     apt-get update
7     apt-get install -y python3 python3-pip git wget
8
9     # Install PyTorch with CUDA support
10    pip3 install torch torchvision torchaudio --index-url https://
    download.pytorch.org/whl/cu121
11
12    # Install additional packages
13    pip3 install numpy pandas matplotlib scikit-learn
14
15    # Cleanup
16    apt-get clean
17    rm -rf /var/lib/apt/lists/*
18
19 %environment
20     export PATH=/usr/local/bin:$PATH
21     export LC_ALL=C
22
23 %labels
24     Description "PyTorch with CUDA 12.1 for HPC cluster"
25
26 %help
27     This container includes PyTorch with GPU support.
28     Use --nv flag to enable GPUs.

```

Build the image:

```

1 apptainer build pytorch-gpu.sif pytorch-gpu.def

```

4.3 Example 3: Conda/Python Environment

Create conda-env.def:

```

1 Bootstrap: docker
2 From: continuumio/miniconda3
3

```

```

4 %post
5     # Update conda and install packages
6     conda update -n base -c defaults conda -y
7     conda create -n myenv python=3.10 -y
8     conda run -n myenv pip install torch pandas numpy
9
10 %environment
11     export PATH=/opt/conda/envs/myenv/bin:$PATH
12     conda activate myenv
13
14 %runscript
15     exec python "$@"

```

Build:

```
1 apptainer build conda-env.sif conda-env.def
```

4.4 Example 4: R Environment for Data Science

Create r-env.def:

```

1 Bootstrap: docker
2 From: rocker/r-ubuntu:24.04
3
4 %post
5     # Install R packages
6     R -e "install.packages(c('tidyverse', 'data.table', 'ggplot2', '
7     caret'), repos='https://cran.rstudio.com/')"
8
9     # Install Python for mixed workflows
10    apt-get update
11    apt-get install -y python3 python3-pip
12    pip3 install pandas numpy
13
14 %environment
15     export LC_ALL=C.UTF-8
16
17 %runscript
18     Rscript "$@"

```

4.5 Example 5: Complete ML Environment (PyTorch + TensorFlow + Jupyter)

Create ml-full.def:

```

1 Bootstrap: docker
2 From: nvidia/cuda:12.1.1-devel-ubuntu22.04
3
4 %post
5     apt-get update
6     apt-get install -y python3 python3-pip python3-venv git wget curl
7     vim
8
9     # PyTorch
10    pip3 install torch torchvision torchaudio --index-url https://
    download.pytorch.org/whl/cu121

```

```

11 # TensorFlow
12 pip3 install tensorflow==2.15.0
13
14 # Jupyter and data science
15 pip3 install jupyterlab pandas numpy matplotlib seaborn scikit-
  learn
16
17 # Development tools
18 pip3 install black flake8 pytest ipython
19
20 apt-get clean
21
22 %environment
23 export PATH=/usr/local/bin:$PATH
24 export PYTHONPATH=/usr/local/lib/python3.10/site-packages
25
26 %runscript
27 exec python3 "$@"

```

Build Commands Reference

Command	Description
apptainer build image.sif image.def	Build from definition file
apptainer build --sandbox sandbox/ image.def	Build writable sandbox (for development)
apptainer build image.sif docker://ubuntu:24.04	Pull from Docker Hub
apptainer build image.sif library://alpine	Pull from Sylabs library
apptainer build --fakeroot image.sif image.def	Build without sudo (requires fakeroot)

5.1 Build Options Explained

- **--sandbox:** Creates a writable directory instead of a read-only .sif file (useful for testing)
- **--fakeroot:** Allows building without sudo (requires kernel support)
- **--force:** Overwrites existing output file
- **--notest:** Skips post-build tests
- **--fix-perms:** Fixes permission issues in the container

Transferring to Your Cluster

Once built on your local machine:

```
1 # Copy to submission node
2 scp mycontainer.sif username@clusterlogin:
```

6.1 Organizing Containers on the Cluster

```
1 # Create a containers directory in your home
2 mkdir -p ~/containers
3
4 # Copy multiple containers
5 scp *.sif username@clusterlogin:~/containers/
```

Running container on the Cluster

7.1 Basic Slurm Job Script with Container

Create a job script `container_job.slurm`:

```
1 #!/bin/bash
2 #SBATCH --job-name=container_job
3 #SBATCH --output=%j.out
4 #SBATCH --error=%j.err
5 #SBATCH --time=02:00:00
6 #SBATCH --partition=gpu-P100
7 #SBATCH --gres=gpu:1
8 #SBATCH --cpus-per-task=8
9 #SBATCH --mem=32G
10
11 # === CRITICAL: Change to local SSD ===
12 cd "$TMPDIR" || exit 1
13
14 echo "Job $SLURM_JOB_ID running on $(hostname)"
15 echo "Using container: ~/containers/pytorch-gpu.sif"
16
17 # Copy container to local scratch for faster access
18 cp ~/containers/pytorch-gpu.sif ./
19
20 # Run container with GPU support
21 apptainer exec --nv ./pytorch-gpu.sif python train.py --output ./
    results/
22
23 echo "Job completed at $(date)"
```

7.2 Container Flags Explained

Flag	Purpose
--nv	Enable NVIDIA GPU support (CUDA)
--bind /host/path:/container/path	Mount directories into container

<code>--pwd /workdir</code>	Set working directory inside container
<code>--cleanenv</code>	Clean environment before running
<code>--contain</code>	Disable bind mounts and network (more isolation)

7.3 Example: Mounting Directories

```

1 # Mount current directory and dataset location
2 #!/bin/bash
3 #SBATCH --job-name=pytorch_container
4 #SBATCH --partition=gpu-P100
5 #SBATCH --output=%j.out
6 #SBATCH --error=%j.err
7 #SBATCH --time=00:30:00
8 #SBATCH --gres=gpu:2
9
10
11 echo "=== Container PyTorch Test ==="
12 echo "Job ID: $SLURM_JOB_ID"
13 echo "Node: $(hostname)"
14
15 cd "$TMPDIR" || exit 1
16
17 # Copy files to job Executing node
18 cp "$SLURM_SUBMIT_DIR/simple_mnist.py" ./
19 cp "~/containers/pytorch-23-09.sif" ./
20 cp "$SLURM_SUBMIT_DIR/dataset" ./
21
22
23 # Run in container
24 apptainer exec --nv \
25     --bind $PWD:/workspace \
26     --bind $PWD/dataset:/data
27     $PWD/pytorch-23-09.sif \
28     python /workspace/simple_mnist.py --data /data --epochs 10 --
29     output-dir /workspace/results
30 echo "Job complete"

```

7.4 Example: Different Container Types

```

1 # PyTorch container
2 apptainer exec --nv ~/containers/pytorch.sif python train.py --output
3 ./results/
4
5 # TensorFlow container
6 apptainer exec --nv ~/containers/tensorflow.sif python train_tf.py --
7 output ./results/
8
9 # Ubuntu container (non-GPU)
10 apptainer exec ~/containers/ubuntu.sif bash -c "echo 'Running on CPU'"
11
12 # R container
13 apptainer exec ~/containers/r-env.sif Rscript analysis.R

```

Troubleshooting

8.1 Common Errors and Solutions

Error	Cause	Solution
permission denied	No sudo/fakeroot	Use <code>--fakeroot</code> or install <code>apptainer-suid</code>
no space left	Disk full	Clean up old images, use <code>apptainer cache clean</code>
unable to pull	Network issue	Check proxy settings, use <code>http_proxy</code>
GPU not found	Missing <code>--nv</code> flag	Add <code>--nv</code> to run command

8.2 Cleaning Cache

```

1 # View cache size
2 apptainer cache list
3
4 # Clean all cache
5 apptainer cache clean
6
7 # Clean specific cache
8 apptainer cache clean --older-than 30d

```

Best Practices

1. Pin versions: Always specify exact versions (e.g., `pytorch==2.1.0`) for reproducibility
2. Keep containers small: Clean apt cache, remove unnecessary files in `%post`
3. Use `.sif` files: Avoid writable sandboxes in production
4. Test before scaling: Always test new containers interactively before batch submission
5. Document your containers: Use `%help` section in definition files
6. Version control: Keep definition files in git, not the `.sif` files

Quick Reference Card

```

1 # PULL FROM REGISTRY
2 apptainer build out.sif docker://image:tag
3
4 # BUILD FROM DEFINITION
5 apptainer build out.sif def.def
6
7 # BUILD WITH FAKEROOT
8 apptainer build --fakeroot out.sif def.def

```

```
9
10 # BUILD SANDBOX (writable)
11 apptainer build --sandbox sandbox/ def.def
12
13 # RUN WITH GPU
14 apptainer exec --nv out.sif nvidia-smi
15
16 # INTERACTIVE SHELL
17 apptainer shell --nv out.sif
18
19 # TEST GPU IN CONTAINER
20 apptainer exec --nv out.sif python3 -c "import torch; print(torch.cuda.
    is_available())"
21
22 # CLEAN CACHE
23 apptainer cache clean
24
25 # COPY TO CLUSTER
26 scp out.sif username@clusterlogin:~/containers/
```

Quick Checklist Before Building

Do you have the definition file ready?
Is the base image URL correct?
Are all required packages listed in %post?
Do you have enough disk space (5-10 GB)?
Do you have network access to pull base images?
Have you tested the container interactively?
Is the .sif file transferred to the cluster?

If you answered YES to all, your container is ready!